

EVALUACIÓN PRÁCTICA

Servicio Node.js + Proyecto React 19+

Operaciones CRUD — Gestión de Pagos de Arbitrios Municipales

Mg. Anthony Paul Tavera Ramos | JavaScript Avanzado (100000SI56) | Unidad 2: React

Duración: 90 minutos | Puntaje total: 20 puntos | Evaluación individual

Instrucciones generales

- El examen es individual. Está prohibido compartir código o comunicarse con otros estudiantes.
- Entrega dos carpetas: backend/ (servicio Node.js) y frontend/ (proyecto React 19+), comprimidas en un solo archivo ZIP nombrado con tu código: U[codigo]_Examen_P2.zip
- El servicio Node.js debe ejecutarse en el puerto 3000 y React (Vite) en el puerto 5173.
- En el backend solo puedes usar Express y cors. No se permiten otras dependencias externas.
- El proyecto React 19+ debe usar: TypeScript, Vite, componentes funcionales, Context API + useReducer/useState para estado global, Axios para llamadas HTTP, y React Router para el ruteo.
- Los formularios se implementan con estado controlado simple (useState), sin librerías externas de formularios.
- El docente verificará que el backend responda antes de evaluar el frontend.
- No se aceptan entregas fuera del tiempo establecido ni por correo electrónico.

Distribución del puntaje

Parte	Descripción	Puntos
Parte 1	Servicio Node.js — API REST con Express	8 pts
Parte 2	Proyecto React 19+ — Consumo del API (CRUD)	10 pts
Parte 3	Integración y funcionamiento completo del sistema	2 pts
TOTAL		20 pts

Escenario — Gestión de Pagos de Arbitrios Municipales

Una municipalidad necesita una aplicación web para administrar los pagos de arbitrios municipales (limpieza pública, parques y jardines, serenazgo) realizados por los contribuyentes. Tu tarea es construir el sistema completo: un servicio backend en Node.js que funcione como API REST, y un proyecto React 19+ que lo consuma y permita realizar las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) desde el navegador.

Cada pago de arbitrio registrado tiene los siguientes atributos:

Atributo	Tipo	Descripción / Restricción
id	number	Identificador único. Lo genera el backend automáticamente.
contribuyente	string	Nombre del contribuyente. Campo requerido, mínimo 2 caracteres.
codigoPredio	string	Código del predio asociado. Ej: 'P-00123', 'P-00456'.
tipoArbitrio	string	Tipo de arbitrio. Ej: 'Limpieza Pública', 'Parques y Jardines', 'Serenazgo'.
monto	number	Monto pagado en soles. Valor mínimo: 0.
pagado	boolean	true si el pago fue registrado como efectuado; false si está pendiente.

La siguiente figura muestra la arquitectura del sistema que debes implementar:

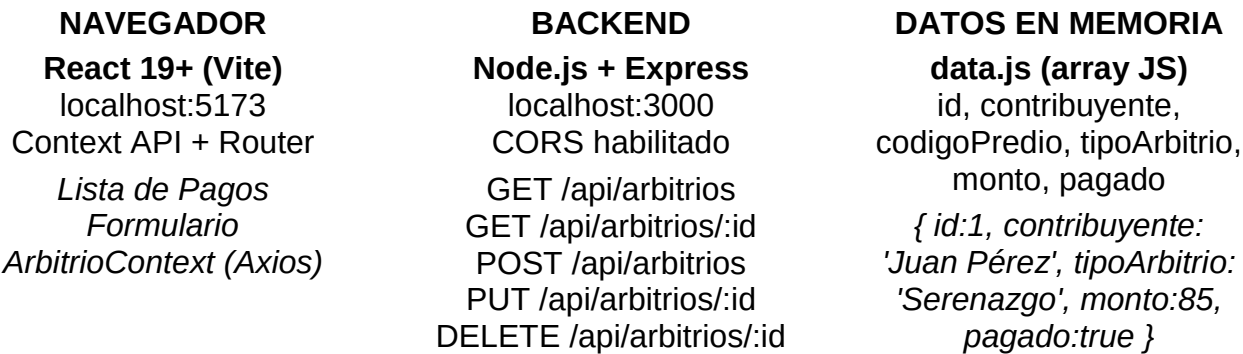


Figura 1. Arquitectura del sistema: navegador React ↔ API Node.js ↔ datos en memoria.

PARTE 1 Servicio Node.js — API REST con Express**8 puntos****1.2 Implementación de los endpoints CRUD**

En el archivo principal del servidor (index.js), implementa los cinco endpoints de la API REST. Todos los endpoints deben devolver respuestas en formato JSON y utilizar los códigos de estado HTTP correctos. El servidor debe tener CORS habilitado para que React pueda consumirlo sin restricciones de origen.

	Método	Ruta	Descripción
a	GET	/api/arbitrios	Devuelve el array completo de pagos de arbitrios.
b	GET	/api/arbitrios/:id	Busca un pago por id. Si no existe, responde con estado 404.
c	POST	/api/arbitrios	Recibe los datos del nuevo pago en el cuerpo de la petición, genera el id automáticamente y lo agrega al array. Responde con estado 201.
d	PUT	/api/arbitrios/:id	Recibe los datos actualizados en el cuerpo, localiza el pago por id y reemplaza sus campos. Si no existe, responde con estado 404.
e	DELETE	/api/arbitrios/:id	Elimina el pago con el id indicado del array. Si no existe, responde con estado 404.

⚠ Los códigos de estado HTTP son parte de la evaluación: 200 OK para lecturas exitosas, 201 Created para creación, 404 Not Found cuando el pago no existe.

PARTE 2 Proyecto React 19+ — Consumo del API**10 puntos**

Crea el proyecto React 19+ con TypeScript y Vite en una carpeta llamada frontend/. La aplicación consume la API REST del backend mediante Axios. El estado de los pagos de arbitrios se gestiona con Context API (useContext + useReducer o useState dentro del Provider). Todos los componentes son funcionales. No se permite el uso de variables globales ni localStorage para almacenar datos.

2.1 Interfaz TypeScript — modelo Arbitrio**1 punto**

Crea la interfaz TypeScript que represente la estructura de un pago de arbitrio. Debe reflejar exactamente los seis atributos del escenario con sus tipos correctos. Ubícala en la carpeta types/ (archivo Arbitrio.ts).

2.2 ArbitrioContext — Axios y estado global**3 puntos**

El contexto debe usar Axios para las peticiones HTTP, mantener el array de pagos en estado interno y exponerlo mediante un Provider junto con las funciones para manipularlo, de modo que los componentes lo consuman con un hook personalizado (useArbitrios) sin poder mutar el array directamente. El contexto debe implementar los siguientes cinco métodos:

	Tarea	Pts
a	listar() — realiza GET al endpoint de lista y actualiza el estado con los datos recibidos.	0.5 pts
b	obtener(id) — realiza GET al endpoint de un pago y retorna una Promise con el resultado.	0.5 pts
c	crear(arbitrio) — realiza POST con los datos del nuevo pago y llama a listar() al completarse.	0.5 pts
d	actualizar(id, arbitrio) — realiza PUT con los datos actualizados y llama a listar() al completarse.	0.5 pts
e	eliminar(id) — realiza DELETE y llama a listar() al completarse.	0.5 pts
f	El estado expuesto por el hook useArbitrios refleja automáticamente los cambios tras cada operación.	0.5 pts

2.3 Componente Lista — tabla de pagos**3 puntos**

Crea el componente Lista en components/Lista/. Este componente es la vista principal de la aplicación. Al inicializarse debe cargar todos los pagos desde el contexto y mostrarlos en una tabla. Desde esta vista el estudiante podrá navegar al formulario o eliminar registros.

	Tarea	Pts
a	Al inicializar el componente (useEffect), llama al método listar() del contexto para cargar los pagos en el estado.	0.5 pts

b	Muestra todos los pagos en una tabla recorriendo el array con <code>.map()</code> y key por id. La tabla debe incluir una columna por cada atributo del modelo.	1 pt
c	Cada fila de la tabla debe tener un botón Eliminar que llame al método <code>eliminar()</code> del contexto. La tabla debe actualizarse automáticamente sin recargar la página.	0.5 pts
d	Cada fila debe tener un botón Editar que navegue a la ruta del formulario pasando el id del pago como parámetro (<code>useNavigate</code>).	0.5 pts
e	En el encabezado de la vista debe haber un botón Nuevo Pago que navegue al formulario en modo creación.	0.5 pts

2.4 Componente Formulario — crear y editar

3 puntos

Crea el componente Formulario en `components/Formulario/`. Este único componente atiende dos modos: creación de un nuevo pago y edición de uno existente. El modo se determina leyendo el parámetro de la ruta con `useParams`: si el parámetro es 'nuevo' el formulario aparece vacío; si es un id numérico, el formulario se precarga con los datos del pago correspondiente.

	Tarea	Pts
a	Implementa el formulario con estado controlado simple (<code>useState</code>) para cada campo: contribuyente, <code>codigoPredio</code> , <code>tipoArbitrio</code> , monto y pagado.	0.5 pts
b	Aplica validaciones manuales: contribuyente y <code>codigoPredio</code> son requeridos con mínimo 2 caracteres; monto es requerido con valor mínimo de 0. Muestra mensajes de error bajo cada campo cuando el valor es inválido y el campo fue tocado.	0.5 pts
c	En modo creación (parámetro 'nuevo'): al enviar el formulario válido llama a <code>crear()</code> del contexto.	0.5 pts
d	En modo edición (parámetro numérico): al inicializar llama a <code>obtener(id)</code> para precargar el formulario con los datos actuales del pago. Al enviar el formulario válido llama a <code>actualizar()</code> del contexto.	1 pt
e	Tras guardar exitosamente, el componente navega automáticamente de regreso a <code>/lista</code> . Debe haber también un botón Cancelar que navegue a <code>/lista</code> sin guardar.	0.5 pts

2.5 Configuración de rutas — App.tsx requerido

- Ruta raíz (`/`) que redirija a `/lista` (componente `Navigate`).
- Ruta `/lista` que cargue el componente `Lista`.
- Ruta `/formulario/:id` que cargue el componente `Formulario`. El parámetro `:id` será 'nuevo' para crear o un número para editar.
- Ruta comodín (`*`) que redirija a `/lista`. Debe ser la última ruta del arreglo de rutas.

PARTE 3 Integración y funcionamiento completo**2 puntos**

Esta parte evalúa que el sistema funcione de forma integrada con el backend corriendo en el puerto 3000 y el frontend en el puerto 5173. El docente verificará cada operación directamente en el navegador con ambos servidores activos.

Operación	Criterio de verificación	Pts
Listar	Los pagos de muestra del backend aparecen en la tabla al ingresar a /lista.	0.5 pts
Crear	Completar el formulario en modo nuevo y guardar agrega el nuevo pago a la tabla sin recargar la página.	0.5 pts
Editar	Modificar un campo desde el formulario de edición actualiza el dato en la tabla al regresar a /lista.	0.5 pts
Eliminar	El botón Eliminar de una fila la quita de la tabla inmediatamente, sin recargar la página.	0.5 pts

Rúbrica de calificación

Ítem	Criterio principal	Pts
1.1	Proyecto Node.js inicializado, data.js con 3 pagos exportados, servidor arranca en :3000	2
1.2a	GET /api/arbitrios devuelve array completo con status 200	1
1.2b	GET /api/arbitrios/:id devuelve el pago correcto o status 404	1
1.2c	POST crea el pago con id autogenerado y responde status 201	2
1.2d	PUT actualiza los campos del pago existente o responde status 404	1
1.2e	DELETE elimina el pago del array o responde status 404	1
2.1	Interfaz Arbitrio con los 6 atributos y tipos correctos	1
2.2	ArbitrioContext: estado + hook personalizado + 5 métodos con Axios + listar() se invoca al completar	3
2.3	Lista: carga inicial, renderizado con .map() y key, botones Eliminar/Editar/Nuevo operativos	3
2.4	Formulario: estado controlado, validaciones manuales, modo crear, modo editar con precarga, redirección	3
3	Ciclo CRUD completo funciona en el navegador con ambos servidores activos	2
TOTAL		20

⊗ **ENTREGA FINAL:** comprime backend/ y frontend/ en un único archivo ZIP nombrado U[codigo]_Examen_P2.zip y súbelo al aula virtual antes de que expire el tiempo. No se aceptan entregas tardías ni por otros medios.